Chapter 7:
Neural Networks

Assoc. Prof. Dr. Duong Tuan Anh
HCMC University of Technology
July 2015

Outline

- 1. Neural Networks Representation
- 2. Appropriate problems for Neural Network Learning
- 3. Perceptrons
- 4. Multilayer Networks and the Backpropagation algorithm
- 5. Remarks on the Backpropagation algorithm
- 6. Neural network application development
- 7. Benefits and Limitations of Neural networks
- 8. Neural network applications
- 9. Time series prediction using Neural Networks

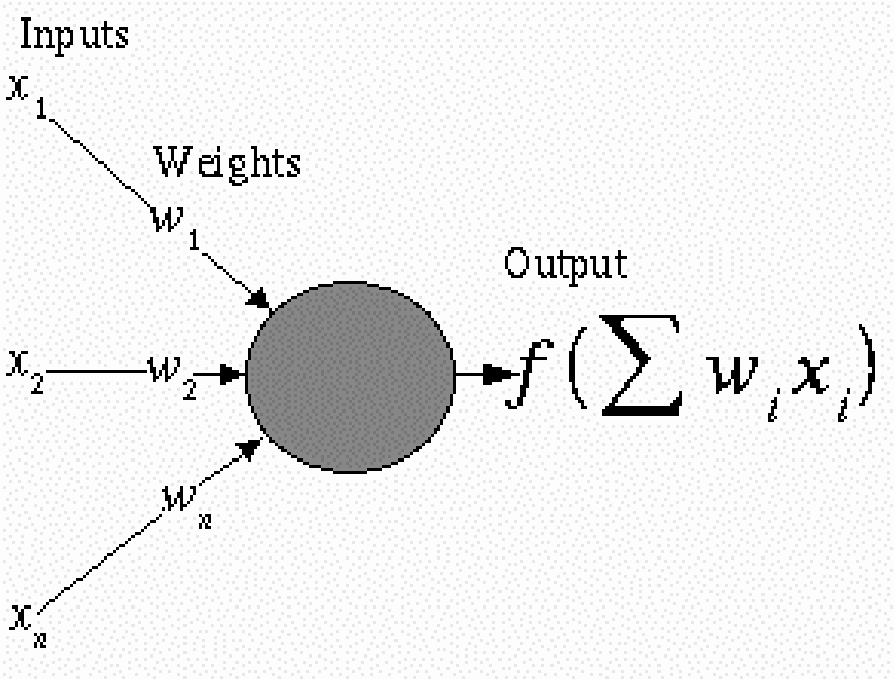
1. NEURAL NETWORK REPRESENTATION

- An ANN is composed of processing elements called or ***perceptrons***, organized in different ways to form the network's structure.

Processing Elements

- An ANN consists of perceptrons. Each of the perceptrons receives inputs, processes inputs and delivers a single output.

The input can be raw input data or the output of other perceptrons. The output can be the final result (e.g. 1 means *yes*, 0 means *no*) or it can be inputs to other perceptrons.



The network

- Each ANN is composed of a collection of *perceptrons* grouped in layers. A typical structure is shown in Fig.2.

Note the three layers: input, intermediate (called the ***hidden layer***) and output.

Several hidden layers can be placed between the input and output layers.

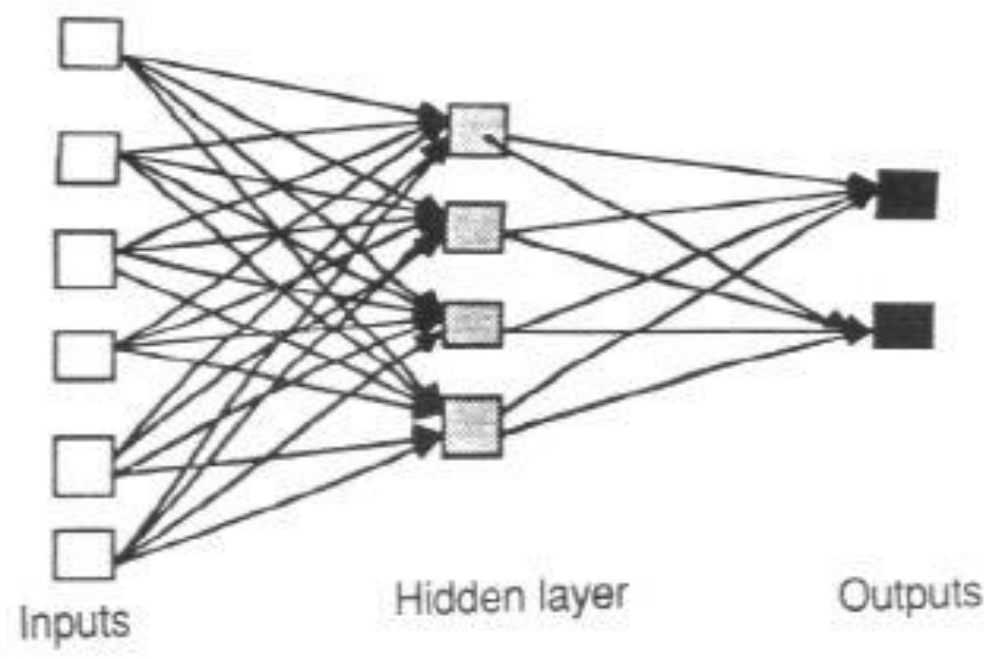


Figure 2

2. Appropriate Problems for Neural Network

- ANN learning is well-suited to problems in which the training data corresponds to noisy data. It is also applicable to problems for which more symbolic representations are used.
- The **back-propagation** (BP) algorithm is the most commonly used ANN learning technique. It is appropriate for problems with the characteristics:
  - Input is high-dimensional discrete or real-valued (e.g. raw sensor input)
  - Output is discrete or real valued
  - Output is a vector of values
  - Possibly noisy data
  - Long training times accepted
  - Fast evaluation of the learned function required.
  - Not important for humans to understand the weights
- Examples:
  - Speech phoneme recognition
  - Image classification
  - Financial prediction

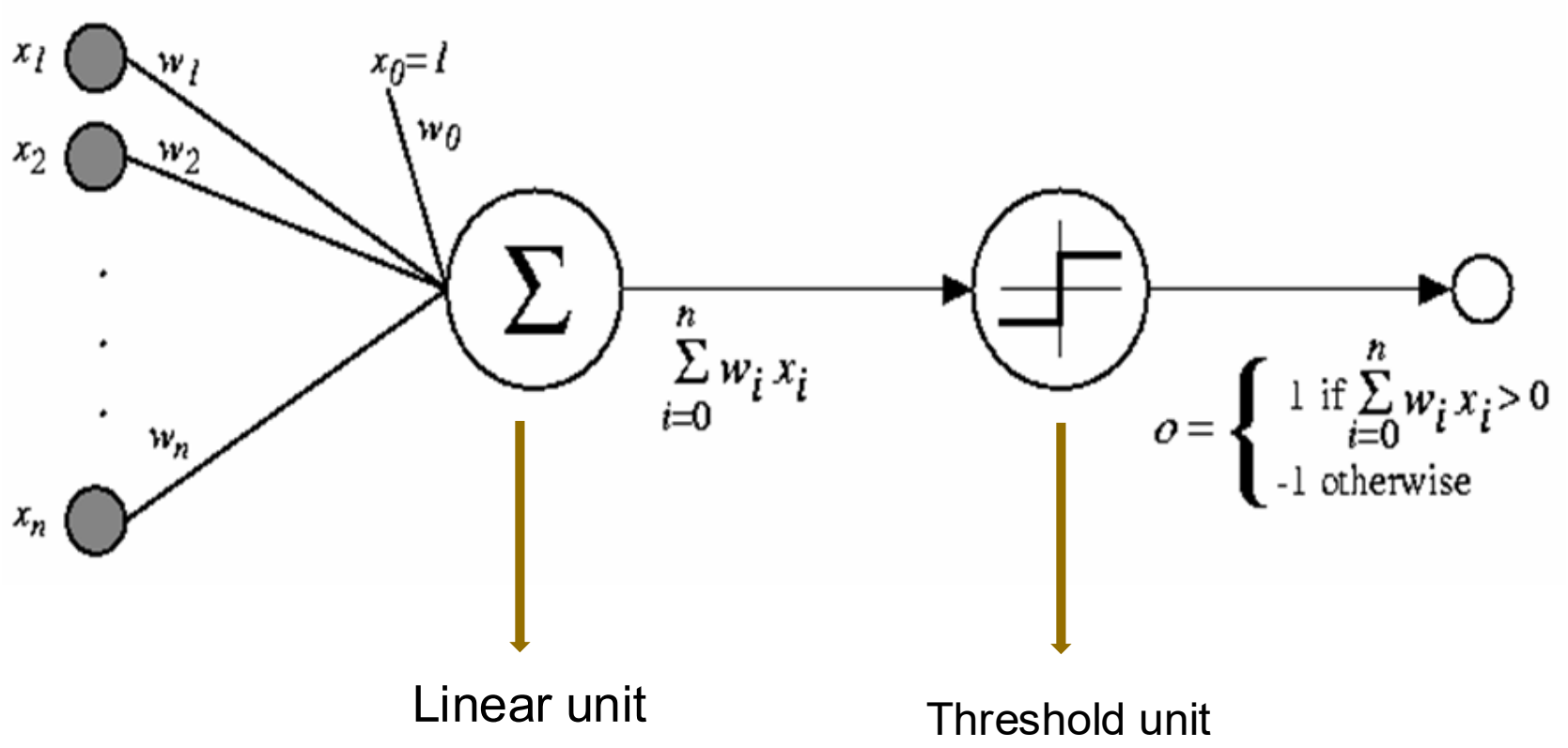
3. PERCEPTRONS

- A perceptron takes a vector of real-valued inputs, calculates a linear combination of these inputs, then outputs
  - a 1 if the result is greater than some threshold
  - –1 otherwise.
- Given real-valued inputs x_1 through x_n, the output $o(x_1, \ldots, x_n)$ computed by the perceptron is

$$o(x_1, \ldots, x_n) = \begin{cases} 1 & \text{if } w_0 + w_1 x_1 + \ldots + w_n x_n > 0 \\ -1 & \text{otherwise} \end{cases}$$

  where w_i is a real-valued constant, or ***weight***.
- Notice the quantify ($-w_0$) is a ***threshold*** that the weighted combination of inputs $w_1 x_1 + \ldots + w_n x_n$ must surpass in order for perceptron to output a 1.

Figure 3. A perceptron



- To simplify notation, we imagine an additional constant input $x_0 = 1$, allowing us to write the above inequality as

$$\sum_{i=0}^{n} w_i x_i > 0$$

or in vector form as

$$\vec{w}.\vec{x} > 0$$

For brevity, we will sometimes write the perceptron function as

$$o(\vec{x}) = \mathrm{sgn}(\vec{w}.\vec{x})$$

- Learning a perceptron involves choosing values for the weights $w_0, w_1, \ldots, w_n$.

Representation Power of Perceptrons

- We can view the perceptron as representing a hyperplane **decision surface** in the *n*-dimensional space of instances (i.e. points). The perceptron outputs a 1 for instances lying on one side of the hyperplane and outputs a –1 for instances lying on the other side, as in Figure 4.
- The equation for this **decision hyperplane** is

$$\vec{w} \cdot \vec{x} = 0$$

Some sets of positive and negative examples cannot be separated by any hyperplane. Those that can be separated are called *linearly separated set of examples.*

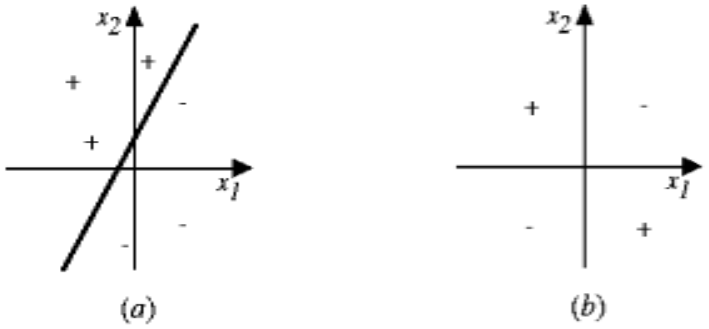


Figure 4. Decision surface

How to train a perceptron

- Although we are interested in learning networks of many interconnected units, let us begin by understanding how to learn the weights for *a single perceptron*.
- Here learning is to determine a **weight vector** that causes the perceptron to produce the correct **+1** or **–1** for each of the given training examples.
- Several algorithms are known to solve this learning problem. Here we consider two:
  - the **perceptron training rule** and
  - the **delta rule**.

Perceptron training rule

- One way to learn an acceptable weight vector is **to *begin with random weights*, then iteratively apply the perceptron to each training example, *modifying the perceptron weights* whenever it misclassifies an example.** This process is repeated, iterating through the training examples as many as times needed until the perceptron *classifies* all training examples correctly.

- Weights are modified *at each step* according to the perceptron training rule, which revises the weight w_i associated with input x_i according to the rule.

$$w_i \leftarrow w_i + \Delta w_i$$

where $$\Delta w_i = \eta(t - o)x_i$$

- Here:

  t is target output value for the current training example

  o is perceptron output

  η is small constant (e.g., 0.1) called *learning rate*

Perceptron training rule (cont.)

- The role of the **learning rate** is to moderate the degree to which weights are changed at each step. It is usually set to some small value (e.g. 0.1) and is sometimes made to decrease as the number of weight-tuning iterations increases.
- We can prove that the algorithm will converge
  - If training data is *linearly separable*
  - and η sufficiently small.
- If the data is *not linearly separable*, convergence is not assured.

The Delta Rule (Gradient Descent)

- Although the **perceptron training rule** finds a successful weight vector when the training examples are linearly separable, it can fail to converge if the examples are *not linearly separatable*. A second training rule, called the **delta rule**, is designed to overcome this difficulty.
- The key idea of *delta rule*: to use ***gradient descent*** to search the space of possible weight vectors to find the weights that best fit the training examples.
- The delta rule is important because it provides the *basis* for the *back-propagation* algorithm, which can learn networks with many interconnected units.
- The delta training rule: considering the task of training an un-thresholded perceptron, that is a *linear unit*, for which the output o is given by:

$$o = w_0 + w_1 x_1 + \cdots + w_n x_n \qquad (1)$$

- Thus, a *linear unit* corresponds to the first stage of a perceptron, without the threshold.

Delta rule

- In order to derive a weight learning rule for linear units, let specify a measure for the ***training error*** of a weight vector, relative to the training examples.
- The **Training Error** can be computed as the following squared error

$$E[\vec{w}] \equiv \frac{1}{2} \sum_{d \in D} (t_d - o_d)^2 \qquad (2)$$

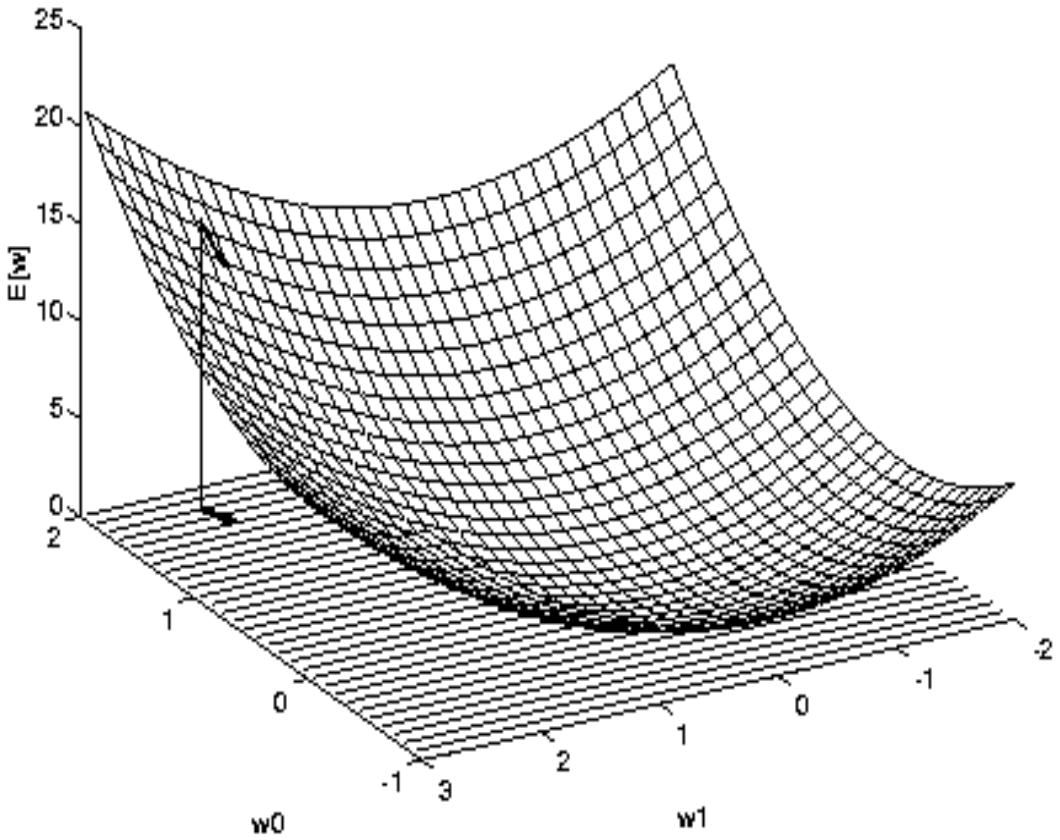
where D is set of training examples, t_d is the target output for the training example d and o_d is the output of the linear unit for the training example d.

Here we characterize E as ***a function of weight vector*** because the linear unit output O depends on this weight vector.

Hypothesis Space

- To understand the *gradient descent* algorithm, it is helpful to visualize the entire space of possible weight vectors and their associated *E* values, as shown in Figure 5.
  - Here the axes w_o, w_1 represents possible values for the two weights of a simple linear unit. The w_o, w_1 plane represents the entire **hypothesis space**.
  - The vertical axis indicates the error *E* relative to some fixed set of training examples. The **error surface** shown in the figure summarizes the desirability of every weight vector in the hypothesis space.
- For linear units, this *error surface* must be **parabolic** with a single global minimum. And we desire a weight vector with this minimum.

Figure 5. The error surface



How can we calculate the direction of steepest descent along the error surface? This direction can be found by computing the derivative of *E* w.r.t. each component of the vector *w*.

Derivation of the Gradient Descent Rule

- This vector derivative is called the *gradient* of E with respect to the vector $\langle w_0,\ldots,w_n\rangle$, written ∇E .

$$\nabla E[\vec{w}] \equiv \left[\frac{\partial E}{\partial w_0}, \frac{\partial E}{\partial w_1}, \cdots \frac{\partial E}{\partial w_n}\right] \quad (3)$$

Notice ∇E is itself a vector, whose components are the partial derivatives of E with respect to each of the w_i.

When interpreted as a vector in weight space, the gradient specifies the **direction** that produces the **steepest increase** in E. The negative of this vector therefore gives the direction of **steepest decrease**.

Since the gradient specifies the direction of **steepest increase** of E, the training rule for gradient descent is

$$\vec{w} \leftarrow \vec{w} + \Delta\vec{w}$$

$$\Delta\vec{w} = -\eta\nabla E[\vec{w}] \quad (4)$$

- Here η is a positive constant called the ***learning rate***, which determines the step size in the gradient descent search.
- The *negative sign* is present because we want to move the weight vector in the direction that decreases *E*. This training rule can also be written in its component form

$$w_i \leftarrow w_i + \Delta w_i$$

where

$$\Delta w_i = -\eta \frac{\partial E}{\partial w_i} \qquad (5)$$

which makes it clear that steepest descent is achieved by altering each component w_i of weight vector in proportion to $\partial E/\partial w_i$.

The vector of $\partial E/\partial w_i$ derivatives that form the gradient can be obtained by differentiating *E* from Equation (2), as

$$
\begin{aligned}
\frac{\partial E}{\partial w_i} &= \frac{\partial}{\partial w_i}\frac{1}{2}\sum_d (t_d - o_d)^2 \\
&= \frac{1}{2}\sum_d \frac{\partial}{\partial w_i}(t_d - o_d)^2 \\
&= \frac{1}{2}\sum_d 2(t_d - o_d)\frac{\partial}{\partial w_i}(t_d - o_d) \\
&= \sum_d (t_d - o_d)\frac{\partial}{\partial w_i}(t_d - \vec{w}\cdot\vec{x_d}) \\
\frac{\partial E}{\partial w_i} &= \sum_d (t_d - o_d)(-x_{i,d}) \qquad (6)
\end{aligned}
$$

where x_{id} denotes the single input component x_i for the training example *d*.

We now have an equation that gives $\partial E/\partial w_i$ in terms of the linear unit inputs x_{id}, output o_d and the target value t_d associated with the training example. Substituting Equation (6) into Equation (5) yields the **weight update rule** for gradient descent.

$$\Delta w_i = \eta \sum_{d \in D} (t_d - o_d)\, x_{id} \qquad (7)$$

- The **gradient descent** algorithm for training linear units is as follows:
  - Pick an initial random weight vector. Apply the linear unit to all training examples, then compute Δw_i for each weight according to Equation (7). Update each weight w_i by adding Δw_i , them repeat the process. The algorithm is given in Figure 6.
- Because the **error surface** contains only a single global minimum, this algorithm will converge to a weight vector with minimum error, regardless of whether the training examples are linearly separable, given a sufficiently small η is used.
- If η is too large, the **gradient descent** search runs the risk of overstepping the minimum in the error surface rather than settling into it. For this reason, one common modification to the algorithm is to **gradually reduce** the value of η as the number of gradient descent steps grows.

GRADIENT-DESCENT($training\_examples, \eta$)

Each training example is a pair of the form $\langle \vec{x}, t \rangle$, *where* $\vec{x}$ *is the vector of input values, and* t *is the target output value.* η *is the learning rate (e.g., .05).*

- Initialize each w_i to some small random value
- Until the termination condition is met, Do
  - Initialize each Δw_i to zero.
  - For each $\langle \vec{x}, t \rangle$ in *training_examples*, Do
    - Input the instance $\vec{x}$ to the unit and compute the output o
    - For each linear unit weight w_i, Do

$$\Delta w_i \leftarrow \Delta w_i + \eta(t - o)x_i \quad (8)$$

  - For each linear unit weight w_i, Do

$$w_i \leftarrow w_i + \Delta w_i \quad (9)$$

Figure 6. Gradient Descent algorithm for training a linear unit.

Stochastic Approximation to Gradient Descent

- The key difficulties in applying gradient descent are:
  - Converging to a local minimum can sometimes be quite slow.
  - If there are multiple local minima in the error surface, then there is no guarantee that the procedure will find the global minimum.
- One common variation on gradient descent to alleviate these difficulties is called *incremental gradient descent* (or *stochastic gradient descent*).
- The key differences between standard gradient descent and stochastic gradient descent are:
  - In standard gradient descent, the error is summed over **all examples** before upgrading weights, whereas in stochastic gradient descent weights are updated upon examining **each training example**.
  - The modified training rule is like the training rule given by Equation (7) except that as we iterate through each example we update the weight according to

$$\Delta w_i = \eta(t - o)\, x_i \qquad (10)$$

where t, o and x_i are the target value, unit output, and the ith input.

- To modify the gradient descent algorithm in Figure 6 to implement this stochastic approximation, Equation $w_i \leftarrow w_i + \Delta w_i$ is simply deleted and Equation $\Delta w_i \leftarrow \Delta w_i + \eta(t - o)x_i$ is replaced by

$$w_i \leftarrow w_i + \eta(t - o)x_i.$$

- One way to view this stochastic gradient descent is to consider a distinct error function defined for each individual training example *d* as follows

$$E_d(\vec{w}) = \frac{1}{2}(t_d - o_d)^2$$

where t_d and o_d are the target value and the unit output value for training example *d*.

We come to the stochastic gradient descent algorithm (Figure. 7)

- Summing over multiple examples in *standard gradient descent* requires more computation per weight update step. On the other hand, because it uses the true gradient, *standard gradient descent* is often used with a larger step size per weight update than *stochastic gradient descent*.

Batch mode Gradient Descent:
Do until satisfied

1. Compute the gradient $\nabla E_D[\vec{w}]$
2. $\vec{w} \leftarrow \vec{w} - \eta \nabla E_D[\vec{w}]$

Incremental mode Gradient Descent:
Do until satisfied

- For each training example d in D
  1. Compute the gradient $\nabla E_d[\vec{w}]$
  2. $\vec{w} \leftarrow \vec{w} - \eta \nabla E_d[\vec{w}]$

$$E_D[\vec{w}] \equiv \frac{1}{2} \sum_{d \in D} (t_d - o_d)^2$$

$$E_d[\vec{w}] \equiv \frac{1}{2}(t_d - o_d)^2 \qquad (11)$$

Figure 7. Stochastic gradient descent algorithm

- *Stochastic gradient descent* (i.e. incremental mode) can sometimes avoid falling into local minima because it uses the various gradient of *E* rather than overall gradient of *E* to guide its search.
- Both stochastic and standard gradient descent methods are commonly used in practice.

Summary

- *Perceptron training rule*
  - Perfectly classifies training data
  - Converge, provided the training examples are linearly separable

- *Delta Rule using gradient descent*
  - Converge asymptotically to minimum error hypothesis
  - Converge regardless of whether training data are linearly separable

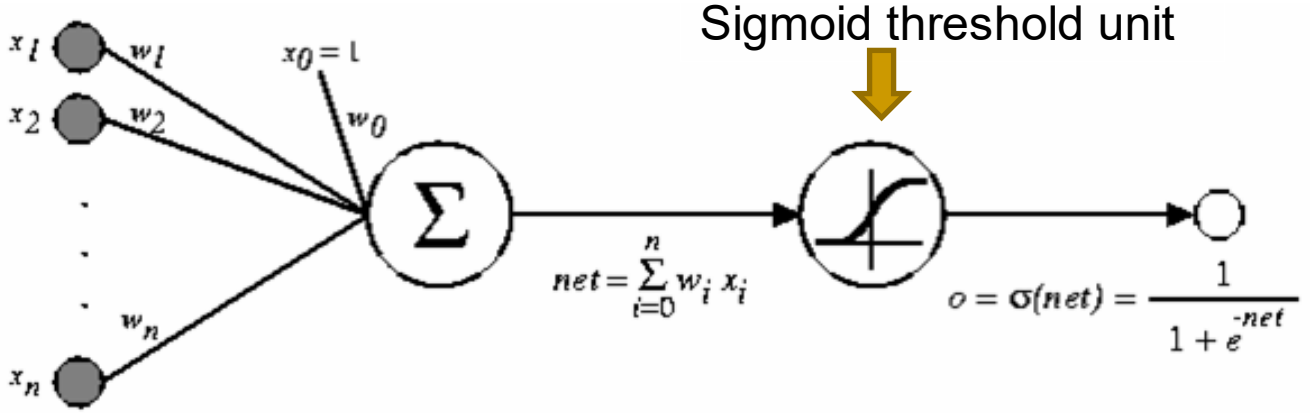
4. MULTILAYER NETWORKS AND THE BACKPROPOGATION ALGORITHM

- Single perceptrons can only express ***linear decision surfaces***. In contrast, the kind of multilayer networks learned by the backpropagation algorithm are capaple of expressing a rich variety of ***nonlinear decision surfaces***.
- This section discusses how to learn such multilayer networks using a *gradient descent* algorithm similar to that discussed in the previous section.

A Differentiable Threshold Unit

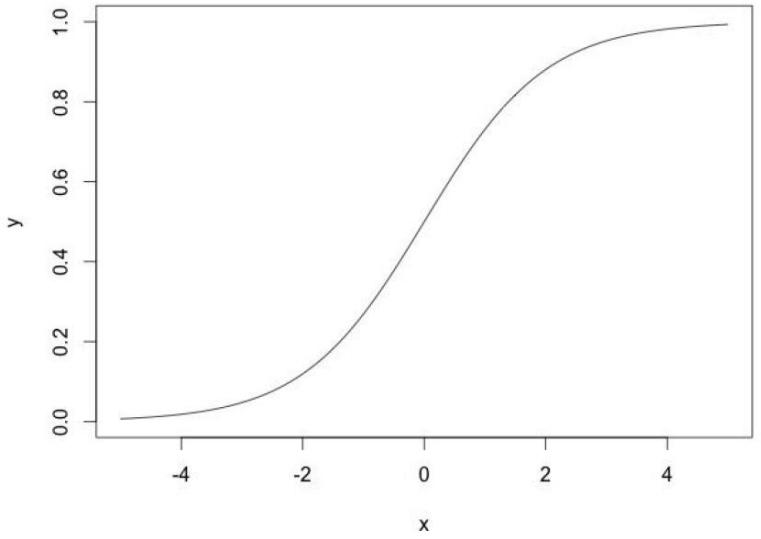
- What type of unit as the basis for multilayer networks ?
  - Perceptron : not differentiable -> can't use gradient descent
  - Linear Unit : multi-layers of linear units -> still produce only linear function
  - Sigmoid Unit : *smoothed*, *differentiable threshold function*

Figure 7. The sigmoid threshold unit.



$\sigma(x)$ is the sigmoid function

$$\frac{1}{1 + e^{-x}} \quad (12)$$



The sigmoid unit

- Like the perceptron, the sigmoid unit first computes a *linear combination* of its inputs, then applies a *threshold* to the result. In the case of sigmoid unit, however, the threshold output is a continuous function of its input.
- The **sigmoid** function $\sigma(x)$ is also called the **logistic function**.
- Interesting property:

$$\frac{d\sigma(x)}{dx} = \sigma(x)(1 - \sigma(x))$$

• Output ranges between 0 and 1, increasing monotonically with its input.

We can derive gradient descent rules to train

• One sigmoid unit

• *Multilayer networks* of sigmoid units $\Rightarrow$ Backpropagation

The Backpropagation (BP)Algorithm

- The BP algorithm learns the weights for a **multilayer network**, given a network with a fixed set of units and interconnections. It employs a **gradient descent** to attempt to minimize the squared error between the network output values and the target values for these outputs.
- Because we are considering networks with *multiple output units* rather than single unit as before, we begin by redefining E to sum the errors over all of the network output units

$$E_d(\vec{w}) = \frac{1}{2} \sum_{k \in outputs} (t_{kd} - o_{kd})^2 \qquad (13)$$

where *outputs* is the set of output units in the network, and t_{kd} and o_{kd} are the target and output values associated with the k-th output unit and training example d.

The Backpropagation Algorithm (cont.)

- The BP algorithm is presented in Figure 8. The algorithm applies to layered feed-forward networks containing 2 layers of sigmoid units, with units at each layer connected to all units from the preceding layer.
- This is an *incremental gradient descent version* of Backpropagation.
- The notation is as follows:
  - x_{ji} denotes the input from node i to unit j, and w_{ji} denotes the corresponding weight.
  - δ_n denotes the error term associated with unit n. It plays a role analogous to the quantity $(t - o)$ in our earlier discussion of the delta training rule.

The Backpropagation algorithm (Fig. 8)

Initialize all weights to small random numbers

Until satisfied, do

for each training example, do

1. Input the training example to the network and compute the network outputs
2. For each output k

$$\delta_k \leftarrow o_k(1 - o_k)(t_k - o_k) \qquad (14)$$

3. For each hidden unit h

$$\delta_h \leftarrow o_h(1 - o_h) \sum_{k \in outputs} w_{kh}\delta_k \qquad (15)$$

4. Update each network weight w_{ji}

$$w_{ji} \leftarrow w_{ji} + \Delta w_{ji}$$

where

$$\Delta w_{ji} = \eta\, \delta_j x_{ji} \qquad (16)$$

- In the BP algorithm, step1 propagates the input forward through the network. And the steps 2, 3 and 4 propagates the **errors** backward through the network.
- The main loop of BP repeatedly iterates over the training examples. For each training example, it applies the ANN to the example, calculates the **error** of the network output for this example, computes the gradient w. r. t. the **error** on the example, then updates all weights in the network. This gradient descent step is iterated until ANN performs acceptably well.
- A variety of termination conditions can be used to halt the procedure.
  - One may choose to halt after a fixed number of iterations through the loop, or
  - once the **error** on the training examples falls below some threshold, or
  - once the **error** on a separate **validation set** of examples meets some criteria.

An Example of Back-propagation algorithm

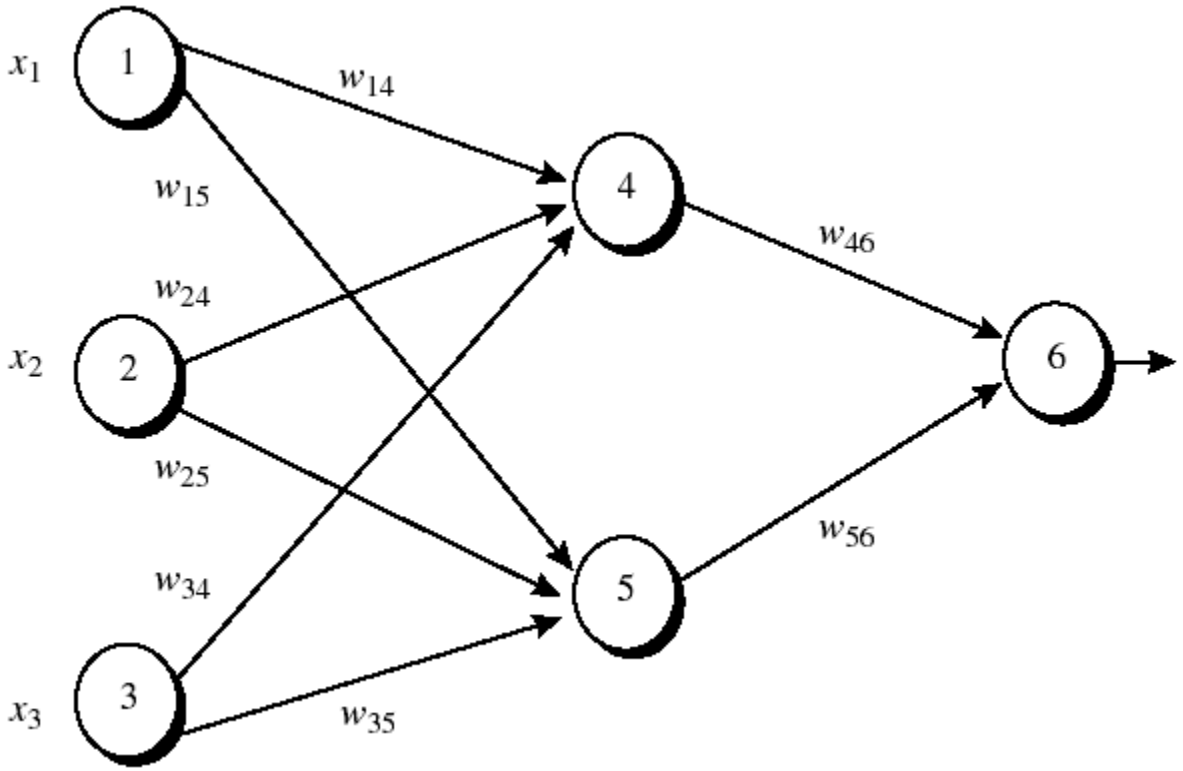


Figure 8: An example of a multilayer feed-forward neural network. Assume that the learning rate η is 0.9 and the first training example, X = (1,0,1) whose class label is 1.

Note: The sigmoid function is applied to hidden layer and output layer.

Table 1: Initial input and weight values

x_1	x_2	x_3	w_{14}	w_{15}	w_{24}	w_{25}	w_{34}	w_{35}	w_{46}	w_{56}	w_{04}	w_{05}	w_{06}
1	0	1	0.2	-0.3	0.4	0.1	-0.5	0.2	-0.3	-0.2	-0.4	0.2	0.1

Table 2: The net input and output calculation

Unit j	Net input I_j	Output O_j
4	0.2 + 0 -0.5 -0.4 = -0.7	$1/(1+e^{0.7})=0.332$
5	-0.3 +0+0.2 +0.2 =0.1	$1/(1+e^{-0.1})=0.525$
6	(-0.3)(0.332)-(0.2)(0.525)+0.1 = -0.105	$1/(1+e^{0.105})=0.474$

Table 3: Calculation of the error at each node

Unit j	δ_j
6	(0.474)(1-0.474)(1-0.474)=0.1311
5	(0.525)(1-0.525)(0.1311)(-0.2)=-0.0065
4	(0.332)(1-0.332)(0.1311)(-0.3)=-0.0087

Table 4: Calculation for weight updating

Weight	New value
w_{46}	-0.3+(0.9)(0.1311)(0.332)= -0.261
w_{56}	-0.2+(0.9)(0.1311)(0.525)= -0.138
w_{14}	0.2 +(0.9)(-0.0087)(1) = 0.192
w_{15}	-0.3 +(0.9)(-0.0065)(1) = -0.306
w_{24}	0.4+ (0.9)(-0.0087)(0) = 0.4
w_{25}	0.1+ (0.9)(-0.0065)(0) = 0.1
w_{34}	-0.5+ (0.9)(-0.0087)(1) = -0.508
w_{35}	0.2 + (0.9)(-0.0065)(1) = 0.194
w_{06}	0.1 + (0.9)(0.1311) = 0.218
w_{05}	0.2 + (0.9)(-0.0065)=0.194
w_{04}	-0.4 +(0.9)(-0.0087) = -0.408

Adding Momentum

- Because BP is a widely used algorithm, many variations have been developed. The most common is to alter the weight-update rule in Step 4 in the algorithm by making the weight update on the *n*th iteration *depend partially on* the update that occurred during the (*n* - 1)-th iteration, as follows:

$$\Delta w_{i,j}(n) = \eta \delta_j x_{i,j} + \alpha \Delta w_{i,j}(n-1) \qquad (18)$$

Here $\Delta w_{i,j}(n)$ is the weight update performed during the *n*-th iteration through the main loop of the algorithm.

- *n*-th iteration update depend on (*n-1*)th iteration
- α: constant between 0 and 1 is called the *momentum.*

Role of momentum term:

- keep the ball rolling through small local minima in the error surface.
- Gradually increase the step size of the search in regions where the gradient is unchanging, thereby speeding convergence.

Derivation of the Backpropagation Rule

Recall from the equation:

$$E_d(\vec{w}) = \frac{1}{2}(t_d - o_d)^2$$

Stochastic gradient descent involves iterating through the training examples one at a time.

In other words, for each training example *d*, every w_{ji} is updated by adding to it Δw_{ji}:

$$\Delta w_{ji} = -\eta \frac{\partial E_d}{\partial w_{ji}} \quad \textbf{(21)}$$

where E_d is the error on training example *d*, summed over all ouput units.

$$E_d(\vec{w}) \equiv \frac{1}{2} \sum_{k \in outputs} (t_k - o_k)^2$$

Notation

- x_{ji} = the ith input to unit j
- w_{ji} = the weight associated with the ith input to unit j
- $net_j = \Sigma_i w_{ji} x_{ji}$ (the weighted sum of input for unit j)
- o_j = the output computed by unit j
- t_j = the target output for unit j
- σ = the sigmod function
- *outputs* = the set of units in the final layer of the network
- *Downstream*(j) = the set of units whose immediate inputs include the output of unit j.

Now we derive an expression for $\partial E_d / \partial w_{ji}$ in order to implement the stochastic gradient descent rule in Equation (21).

Derivation of the Backpropagation Rule (cont.)

- To begin, notice that weight w_{ji} can influence the rest of the network through net_j. So, we can use the chain rule to write:

$$\frac{\partial E_d}{\partial w_{ji}} = \frac{\partial E_d}{\partial net_j} \frac{\partial net_j}{\partial w_{ji}} = \frac{\partial E_d}{\partial net_j} x_{ji} \quad (22)$$

Now our remaining task is to derive a convenient expression for $\partial E_d / \partial net_j$.

We consider two cases: (1) the case where unit j is an output unit and (2) the case where j is an internal unit.

Case 1: Training rule for output unit weights.

- Just as w_{ji} can influence the rest of the network only through net_j, net_j can influence the network only through o_j. So, we can use the chain rule again to write:

$$\frac{\partial E_d}{\partial net_j} = \frac{\partial E_d}{\partial o_j}\frac{\partial o_j}{\partial net_j} \qquad \textbf{(23)}$$

To begin, consider the first term in Equation (23)

$$\frac{\partial E_d}{\partial o_j} = \frac{\partial}{\partial o_j}\frac{1}{2}\sum_{k \in outputs}(t_k - o_k)^2$$

The derivatives in the right hand side will be zero for all output units k except when $k = j$.

We have:

$$\frac{\partial E_d}{\partial o_j} = \frac{\partial}{\partial o_j}\frac{1}{2}(t_j - o_j)^2$$

$$= \frac{1}{2}2(t_j - o_j)\frac{\partial(t_j - o_j)}{\partial o_j}$$

$$= -(t_j - o_j) \qquad \textbf{(24)}$$

Next consider the second term in Equation (23). Since $o_j = \sigma(net_j)$, the derivative $\partial o_j / \partial net_j$ is just the derivative of the sigmod function, which we have already noted is equal to $\sigma(net_j)(1- \sigma(net_j))$. Therefore,

$$\frac{\partial o_j}{\partial net_j} = \frac{\partial \sigma(net_j)}{\partial net_j}$$

$$= o_j(1 - o_j) \qquad \textbf{(25)}$$

- Substituting expressions (24) and (25) into (23), we obtain:

$$\frac{\partial E_d}{\partial net_j} = -(t_j - o_j)\ o_j(1 - o_j) \qquad \textbf{(26)}$$

And combining this with Equation (21) and (22), we have the stochastic gradient descent rule for output units

$$\Delta w_{ji} = -\eta \frac{\partial E_d}{\partial w_{ji}} = \eta\ (t_j - o_j)\ o_j(1 - o_j) x_{ji} \qquad \textbf{(27)}$$

Note this training rule is exactly the weight update rule, implemented by Equation (14) and (15) in the Backpropagation algorithm. Furthermore, we can see that δ_k in Equation(14) is equal to the quantity - $\partial E_d / \partial net_k$.

Case 2: Training rule for Hidden Unit Weights

- In the case where j is an hidden unit in the network, the derivation of the training rule for w_{ji} must take into account the indirect ways in which w_{ji} can influence the network outputs and hence E_d.
- For this reason, we will find it useful to refer to the set of all units immediately downstream of unit j in the network. We denote this set of units by *Downstream*(j).
- Notice that net_j can influence the network outputs (and therefore E_d) only through the units in *Downstream*(j). Therefore, we can write

$$\frac{\partial E_d}{\partial net_j} = \sum_{k \in Downstream(j)} \frac{\partial E_d}{\partial net_k} \frac{\partial net_k}{\partial net_j}$$

$$= \sum_{k \in Downstream(j)} -\delta_k \frac{\partial net_k}{\partial net_j}$$

$$= \sum_{k \in Downstream(j)} -\delta_k \frac{\partial net_k}{\partial o_j} \frac{\partial o_j}{\partial net_j}$$

$$= \sum_{k \in Downstream(j)} -\delta_k \, w_{kj} \frac{\partial o_j}{\partial net_j}$$

$$= \sum_{k \in Downstream(j)} -\delta_k \, w_{kj} \, o_j(1 - o_j)$$

Rearranging terms and using δ_j to denote - $\partial E_d / \partial net_j$, we have

$$\delta_j = o_j(1 - o_j) \sum_{k \in Downstream(j)} \delta_k \, w_{kj}$$

and $\Delta w_{ji} = \eta \, \delta_j \, x_{ji}$

Hyperbolic Tangent Activation function

- Besides *sigmoid*, we can use *tanh* as activation function:

  $tanh(x) = (e^x - e^{-x})/(e^x + e^{-x})$

- Output ranges between -1 and 1, increasing monotonically with its input.
- Property: $tanh'(x) = 1 - [tanh(x)]^2$.

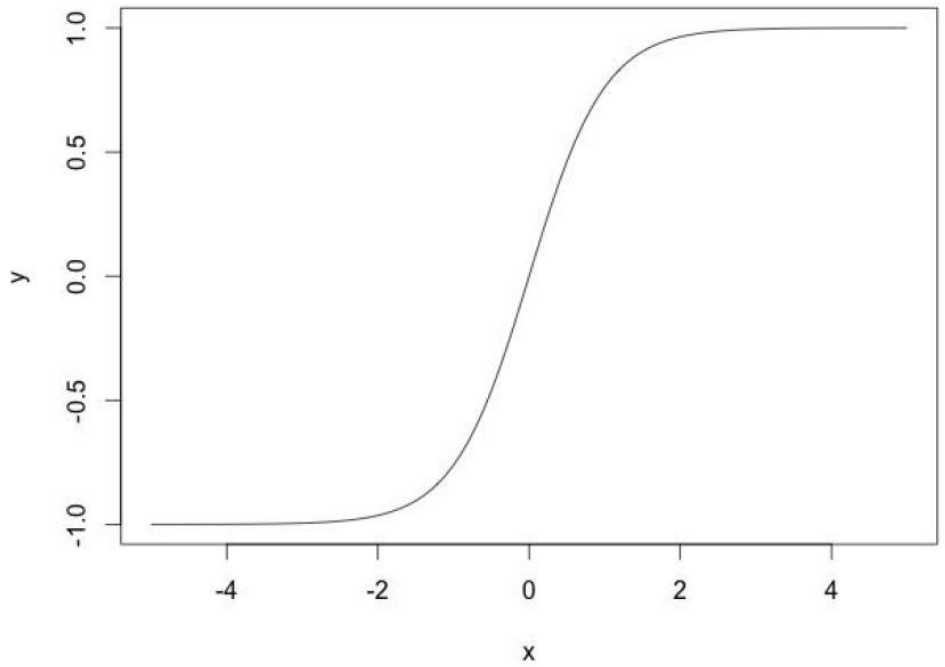


Fig. 9.

5. REMARKS ON THE BACKPROPAGATION ALGORITHM

- **Convergence and Local Minima**
  - Gradient descent to some local minimum
    - Perhaps not global minimum...
  - Heuristics to alleviate the problem of local minima
    - Add momentum
    - Use *stochastic gradient descent* rather than true gradient descent.
    - Train multiple nets with different initial weights using the same data.

6. NEURAL NETWORK APPLICATION DEVELOPMENT

The development process for an ANN application has eight steps.

- *Step 1:* (*Data collection*) The data to be used for the training and testing of ANN are collected. We have to consider that the particular problem is amenable to ANN solution and that adequate data exist and can be obtained.
- *Step 2:* (*Training and testing data separation*) The available data are divided into **training** and **testing** data sets. For a moderately sized data set, 80% of the data are randomly selected for training, 10% for testing, and 10% secondary testing.
- *Step 3:* (*Network architecture*) A network architecture and a ***learning method*** (training algorithm) are selected. Important considerations are the exact number of nodes and the number of layers.

- *Step 4:* (*Parameter tuning and weight initialization*) There are parameters for tuning ANN to the desired learning performance. Part of this step is initialization of the network weights and parameters, followed by modification of the parameters as training performance feedback is received.
  - Often, the initial values are important in determining the effectiveness and length of training.
- *Step 5:* (*Data transformation*) Transforms the application data into the type and format required by the ANN.
- *Step 6:* (*Training*) Training is conducted iteratively by presenting input and known output data to the ANN. The ANN computes the outputs and adjusts the weights until the computed outputs are within an acceptable tolerance of the known outputs for the input cases.

- *Step 7: (Testing)* Once the training has been completed, it is necessary to test the network.
  - The **testing** examines the performance of ANN using the derived weights by measuring the ability of the network to classify the testing data correctly.
- *Step 8:* (*Implementation*) Now a stable set of weights are obtained.
  - Now ANN can reproduce the desired output given inputs like those in the training set.
  - The ANN is ready to use as a *stand-alone system* or as part of another software system where new input data will be presented to it and its output will be a recommended decision.

7. BENEFITS AND LIMITATIONS OF NEURAL NETWORKS

6.1 Benefits of ANNs

- *Usefulness for pattern recognition, classification, generalization, abstraction and interpretation of imcomplete and noisy inputs.* (e.g. handwriting recognition, image recognition, voice and speech recognition, weather forecasing).
- *Providing some characteristics to problem solving* that are difficult to simulate using the logical, analytical techniques of expert systems and standard software technologies.
- *Ability to solve new kinds of problems*. ANNs are particularly effective at solving problems whose solutions are difficult to define. This opened up a new range of decision support applications formerly either difficult or impossible to computerize.

- *Robustness*. ANNs tend to be more robust than their conventional counterparts. They have the ability to cope with imcomplete or fuzzy data. ANNs can be very *tolerant of faults* if properly implemented.
- *Fast processing speed*. Because they consist of a large number of massively interconnected processing units, all operating in parallel on the same problem, ANNs can potentially operate at considerable speed (when implemented on parallel processors).
- *Flexibility and ease of maintenaince*. ANNs are very flexible in adapting their behavior to new and changing environments. They are also easier to maintain, with some having the ability to learn from experience to improve their own performance.

6.2 Limitations of ANNs

- ANNs do not produce an *explicit model* even though new cases can be fed into it and new results obtained.
- *ANNs lack explanation capabilities*. Justifications for results is difficults to obtain because the connection weights usually do not have obvious interpretations.

Network parameters

The following parameters of the ANN are chosen for a closer inspection:

- *The number of input units*:
  - The number of input units determines the number of periods the ANN "looks into the past" when predicting the future. The number of input units is equivalent to the size of the input window.
  - The number of input units is equivalent to the number of attributes of the input sample.
- *The number of output units: depends on the number of classes*
- *The number of hidden units*: Whereas it has been shown that **one** hidden layer is sufficient to approximate continuous function, the number of hidden units necessary is "not known in general". Some examples of ANN architectures that have been used for time series prediction can be 8-8-1, 6-6-1, and 5-5-1.
- **Note**: Ash's method for estimating the number of units in hidden layer.

Network parameters (cont.)

- *The learning rate*: η (0< η < 1) is a scaling factor that tells the learning algorithm how strong the weights of the connections should be adjusted for a given error. A higher η can be used to speed up the learning process, but if η is too high, the algorithm will skip the optimum weights. (The learning rate η is constant across presentations).
- *The momentum parameter* α (0 < α < 1) is another number that affects the gradient descent of the weights: the momentum term is added that keeps the *direction* of the previous step thus avoiding the descent into local minima. (The momentum term is constant across presentations).

8. SOME ANN APPLICATIONS

ANN application areas:

- Face detection
- Face recognition
- Object recognition
- Handwritten character/digit recognition
- Speech recognition
- Image retrieval
- Tax form processing to identify tax fraud
- Enhancing auditing by finding irregularites
- Bankruptcy prediction

ANN applications (cont.)

- Customer credit scoring
- Loan approvals
- Credit card approval and fraud detection
- Financial prediction
- Energy forecasting
- Computer access security (intrusion detection and classification of attacks)
- Fraud detection in mobile telecommunication networks

ANN software tools

- In WEKA, the data mining software, there is ANN tool for classification.
- Spice-Neuro software
- Neural Network Toolbox – MATLAB
- Neural Network in R
- Neural Network in Scikit-Learn

9. Time Series Prediction using Neural network

- Time series prediction: given an existing time series, we model the time series in order to make accurate forecasts

- Example time series
  - Financial (e.g., stock price, exchange rates)
  - Physically observed (e.g., weather, sunspots, river flow)
- Why is it important?
  - Preventing undesirable events by forecasting the event, identifying the circumstances preceding the event, and taking corrective action so the event can be avoided.
  - Forecasting undesirable, yet unavoidable, events to preemptively lessen their impact e.g., flood prediction
  - Profiting from forecasting (e.g., financial markets)

- Why is it difficult?
  - Limited quantity of data (Observed data series sometimes too short to partition)
  - Noise (Erroneous data points, obscuring component)
  - Moving Average
  - Nonstationarity (Fundamentals change over time, nonstationary)
  - Forecasting method selection (Statistics, Artificial intelligence)
- Neural networks have been widely used as time series forecasters: most often these are ***feed-forward networks*** which employ a ***sliding window*** over the input sequence.
- The neural network sees the time series $X_1,\ldots,X_n$ in the form of many mappings of an **input vector** to an **output value**.

Prediction with Neural Networks

- A number of adjoining data points of the time series (the input window $X_{t-s}, X_{t-s-1}, \ldots, X_t$) are used as activation levels for the input units of the input layer.

- The size *s* of the **input window** correspondends to the **number of input units** of the neural network.

How to train a neural network for time series prediction

- In the forward path, these activation levels are propagated over one hidden layer to one output unit. The error used for the **backpropagation learning** algorithm is now computed by comparing the value of the output unit with the transformed value of the time series at time *t+1*. This error is propagated back to the connections between output and hidden layer and to those between hidden and output layer. After all weights have been **updated** accordingly, one presentation has been completed.

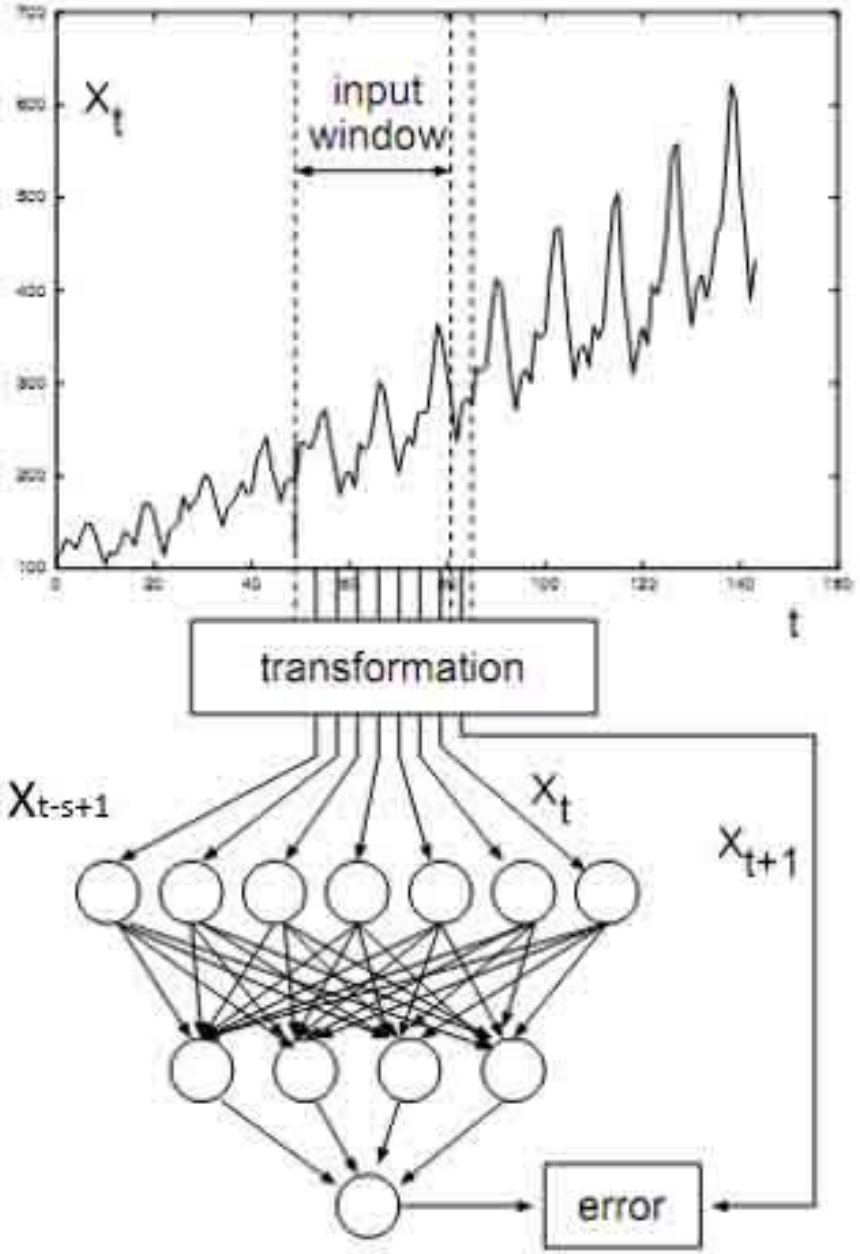


Fig.10 Learning a time series

Prediction with Neural Networks (cont.)

- Training a neural network with backpropagation algorithm usually requires that all representations of the samples (called one *epoch*) are presented **many times**. For examples, the ANN may use 60 to 138 epoches.
- Neural networks can perform time series prediction with high precision since they can capture the **non-linear** characteristics of time series data.

Evaluating Methods of forecasts

- Divide the data into two sections - an *initialization part and a test part*
  - Use the *initialization* data set to train the neural network
  - Use the trained neural network to forecast the *test data* set and determine the forecast errors to evaluate the predictive effectiveness.
  - Evaluate *errors* (MAD, MPE, MSD, MAPE)

Evaluation Methods of Forecasts

- There are three measures of accuracy of the prediction models: MAPE, MAD and MSE.
- For all three measures, the smaller the value, the better the fit of the model.
- Use these statistics to compare the predictive accuracy of the different methods.
- MAPE (Mean Absolute Percentage Error) measure the accuracy of predicted time series values. It expresses accuracy as a percentage.

$$MAPE = \frac{\sum_{t=1}^{n} | (y_t - y'_t) / y_t |}{n} \times 100$$

where y_t is the actual value, y'_t is the predicted value and *n* is the number of observations.

MAPE, MAD, and MSD

- MAD (Mean Absolute Deviation) expresses accuracy in the same units as the data, which help conceptualize the amount of error.

$$MAE = \frac{\sum_{t=1}^{n} | y_t - y'_t |}{n}$$

where y_t is the actual value, y'_t is the predicted value and *n* is the number of observations.

MAPE, MAD, and MSD

- MSE(Mean Squared Error) is a more sensitive measure of an unusually large forecast error than MAD.

$$MSE = \frac{\sum_{t=1}^{n} (y_t - y'_t)^2}{n}$$

where y_t is the actual value, y'_t is the predicted value and *n* is the number of observations.

References

- Tom M. Mitchell, Machine Learning, The McGraw Hill, 1997.
- M. Verleysen, K. Hlavackova, Learning in RBF Networks, *Proc. of Int. Conf. on Neural Networks (ICNN),* Washington, D.C., June 3-6, 1996.
- F. Schwenker, H. A. Kestler, G. Palm, Three learning phases for radial-basis-function networks, *Neural Networks* 14(2001) 439-458.
- N.Benoudjit, C. Archambeau, A. Lendasse, J. Lee, M. Verleysen, Width optimization of the Guassian kernels in RBF Networks, *Proc. of European Symposium on Artificial Neural Networks*, Bruges, Belgium, 24-26 April 2002, pp. 425-432.
- J. Han and M. Kamber, *Data Mining: Concepts and Techniques*, 2nd Edition, Morgan Kaufmann Publishers, 2006

Terminology

- Neural network: *mạng nơ ron*, feed-forward neural network: *mạng nơ ron truyền thẳng*, weight: *trọng số*, hidden layer: *tầng ẩn*, bias: *độ lệch*, linear unit: *đơn vị tuyến tính*, linear combination: *tổ hợp tuyến tính*, threshold: *ngưỡng*, weight vector: *véc tơ trọng số*, perceptron training rule: *luật huấn luyện perceptro*n, linearly separable data: *dữ liệu khả tách một cách tuyến tính*, gradient descent: *suy giảm độ dốc*, incremental gradient descent: *suy giảm độ dốc gia tăng*, error function: *hàm lỗi*, learning rate: *hệ số học*, back-propagation: *lan truyền ngược*, error term: *toán hạng sai số*, sigmoid function: *hàm sigmoid*, transfer function: *hàm truyền*, momentum constant: *hằng số quán tính*